Practical –11

Aim: Write a program for Floyd-Warshal algorithm.

Algorithm:

Input: Number of vertices and an adjancy matrix.
output: A matrix showing the shortest path b/w every pair of vertices
step-1: Start
step-2: Read the number of vertices and the weight matrix
step-3: Initialize a distance matrix equal to the input matrix.
step-4: Set an outer loop for intermediate vertex k from 0 to V-1
step-5: set two inner loops for source i and destination j
from 0 to V-1.
step-6: Update paths using formula: dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]).
step-7: Print the final shortest distance matrix.
step-8: Stop.

Program:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    int dist[10][10];

    cout << "Enter number of vertices: ";
```

```
cin >> n;

cout << "Enter the adjacency matrix:\n";
cout << "(Enter 99999 if there is no direct edge)\n";

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        cin >> dist[i][j];
    }
}

// Floyd-Warshall Algorithm
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][k] != 99999 &&
                dist[k][j] != 99999) {

                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] =
                        dist[i][k] + dist[k][j];
                }
            }
        }
```

```
        }
    }
    cout << "\nShortest Distance Matrix:\n";

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][j] == 99999)
                cout << "INF\t";
            else
                cout << dist[i][j] << "\t";
        }
        cout << endl;
    }

    return 0;
}
```

Output:

```
Enter number of vertices: 4
Enter the adjacency matrix:
(Enter 99999 if there is no direct edge)
0 5 99999 10
99999 0 3 99999
99999 99999 0 1
99999 99999 99999 0

Shortest Distance Matrix:
0   5   8   9
INF 0   3   4
INF INF 0   1
INF INF INF 0
```

Name:M.Ravi Kiran Reddy
Enrollment no : 92460118230

Time complexity :

Time complexity

```
For (int k=0; k<n; k++) {      -> n times
    for (int i=0; i<n; i++) {   -> n times
        For (int j=0; j<n; j++){ -> n times.
            if ( dist [i] [k] + dist [k] [j] < dist [i] [j]){
                dist [i] [j] = dist [i] [k] + dist [k] [j];
            }
        }
    }
}
```

Total T.C:

$= O(n \times n \times n)$

$= O(n^3)$

Best case: $O(n^3)$

Average case: $O(n^3)$

worst case: $O(n^3)$

=> The algorithm uses three nested for loops, each executing n times, regardless of the graph's edge weights.

Conclusion :

Thus, we have successfully implemented the Floyd–Warshall Algorithm in C++ to find the shortest paths between all pairs of vertices in a weighted graph. The algorithm updates the distance matrix by considering each vertex as an intermediate vertex. The best-case, average-case, and worst-case time complexities are all \(O(n^3)\).

Name:M.Ravi Kiran Reddy
Enrollment no : 92460118230